Beyond Heavy Frameworks:
Efficient Deep Learning with Sorix

Democratizing AI Education and Deep Learning in Resource-Constrained Environments

Mitchell Mirano Caro

Official Delegate of Peru

Universidad Nacional Mayor de San Marcos (UNMSM)

Faculty of Mathematical Sciences · Faculty of Physical Sciences

Portfolio: **mitchellmirano.com** · Engine: **sorix.mitchellmirano.com**

World Youth Festival 2026

Ekaterinburg, Russian Federation

About Me: Mitchell Mirano Caro

Scientific Background & Roots

- **Origin**: Chachapoyas, Amazonas, Peru
  Official Delegate of Peru · World Youth Festival 2026.
- **Education (UNMSM, Lima, Peru)**:
  - **M.Sc. Candidate in Solid State Physics** (Physical Sciences).
  - **B.Sc. in Scientific Computing** (Mathematical Sciences).
- **Industry**: Data Scientist at **ARPL Tecnología Industrial**
  MLOps on AWS, CFD-DEM simulations & Industry 4.0.
- **Open-Source Scientific Systems**:
  - **Sorix**: Lightweight deep learning micro-framework.
  - **Tesius**: Multi-model AI chat & unified web workspace.
  - **Qaylla.jl**: HPC numerical analysis in Julia.

Official Profile & CV

mitchellmirano.com

Languages

Spanish (Native) | English
Russian (Basic)

Core Focus

Solid State Physics · AI for Science

Neural Networks: Computational Architecture

A neural network is a **sequence of parameterized differentiable transformations**:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right)$$

- $\mathbf{W}^{(l)}$: Weight matrices scaling representations.
- $\mathbf{b}^{(l)}$: Bias vectors adjusting activation thresholds.
- $\sigma(\cdot)$: Non-linear activation (ReLU, Sigmoid).
- $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$: Scalar error guiding optimization.

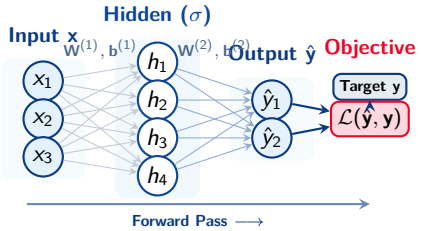

Figure: Forward computational mapping & loss evaluation.

Training: Walking the Loss Surface

Gradient Descent Optimization

Minimize scalar error $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ by stepping along the negative gradient:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

- High-dimensional surfaces feature complex ravines, valleys, and saddle points.
- Hand-derived $\nabla_{\mathbf{w}} \mathcal{L}$ is intractable for large neural architectures.
- **Core Need**: Fast, memory-efficient automatic gradient computation.

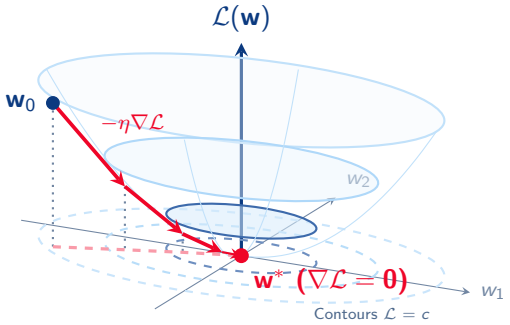

Figure: 3D Convex Loss Bowl & Gradient Trajectory.

The ML Dilemma: Heavy Frameworks vs. Small Models

The Industry Reality

- Mainstream deep learning frameworks are built for datacenter-scale LLMs and massive clusters.
- **The Mismatch**: Most industrial, edge, and educational workloads rely on **compact models** (MLPs, small CNNs, PINNs).
- Multi-gigabytes required just to train or evaluate a lightweight neural network.
- High barrier of entry for students, independent researchers, and low-compute laboratories.

The Practical Barriers

- **Cloud & Serverless**: Crippling cold starts downloading gigabytes of container images on ephemeral workers.
- **Edge & IoT Deployment**: Incapable of running on constrained micro-controllers or low-power embedded hardware.
- **Resource Overhead**: Excessive RAM and energy footprint for straightforward computational tasks.
- **Dependency Fragility**: Heavy, opaque C++ runtimes that complicate cross-platform setups.

The Pedagogical Crisis: Peeling the Black Box

The Black Box Problem in AI Education

- Mainstream frameworks encapsulate execution within opaque, pre-compiled C++ binaries.
- Learners frequently invoke high-level APIs without exposure to Jacobians, gradient routing, or computation graphs.
- Standard documentation prioritizes **API signatures** over foundational mathematical derivations.

The Call for Democratization

- **Mathematical Transparency**: Complete comprehension requires understanding tensor transformations from first principles.
- **Universal Accessibility**: Empowering researchers and students globally to innovate without requiring high-end GPU clusters.

Enter Sorix: Combining minimal compute with complete mathematical openness.

The Genesis of Sorix: Minimalist & Accessible

The Sorix Philosophy

Conceived to train and deploy neural networks with **maximum efficiency** on commodity hardware without binary bloat.

Core Architectural Pillars:

- **PyTorch Parity**: Zero learning curve for researchers.
- **Dual Backend**: Native NumPy (CPU) + CuPy (GPU).
- **Zero C++ Build**: Instant cross-platform deploy.
- **Production Ready**: Native model serialization.

Isolated Environment Size

Framework	CPU Size	GPU Size
Sorix	**54 MB**	**238 MB**
PyTorch	702 MB	6,840 MB
TensorFlow	1,406 MB	1,978 MB

Massive Footprint Advantage

13×–26× **footprint reduction** on CPU and 28× on GPU vs. heavy runtimes.

The Sorix Ecosystem: Ready for Global Use

Distribution & Installation

- **PyPI Standard Package**: `pip install sorix`
- **CUDA GPU Acceleration**: `pip install "sorix[cp13]"`
- **Open-Source Codebase**: Mitchell-Mirano/sorix
- **License**: Permissive Open Source (MIT)

Rigorous Documentation & Tutorials

- **Mathematical Derivations**: Every layer, loss, and backward pass is fully derived with LaTeX proofs.
- **Physics & PINNs**: Interactive Google Colab tutorials for differential equations and deep learning.
- **Official Platform**: Fully accessible at sorix.mitchellmirano.com (scannable via final QR code).

Dual Hardware Engine & Empirical Benchmarks

NumPy on CPU, CuPy on GPU

- Dynamic routing with zero C++ compilation.
- Transparent GPU acceleration:

```
# GPU acceleration (CuPy CUDA)
x = tensor([1.0, 2.0]).to("cuda")

# CPU edge execution (NumPy)
x = tensor([1.0, 2.0]).to("cpu")
```

MNIST Benchmark (5 Epochs):

Framework	Device	Train Time	Inference	Acc
Sorix	**CPU**	**7.36s**	**0.032s**	**97.2%**
PyTorch	CPU	9.21s	0.048s	97.4%
TensorFlow	CPU	17.14s	0.186s	96.8%
Sorix	**GPU**	**6.21s**	**0.015s**	**97.6%**
PyTorch	GPU	4.04s	0.025s	97.6%

Sorix CPU training is 20% faster than PyTorch with the lowest batch latency.

PyTorch Compatibility: Zero Learning Curve

Canonical Deep Learning Pipeline in Sorix

```
from sorix.nn import Sequential, Linear, ReLU, MSELoss
from sorix.optim import SGD

# 1. Modular Model Definition
model = Sequential(
    Linear(784, 128),
    ReLU(),
    Linear(128, 10)
)
criterion = MSELoss()
optimizer = SGD(model.parameters(), lr=0.01)

# 2. Canonical Training Iteration
for epoch in range(epochs):
    optimizer.zero_grad()
    loss = criterion(model(X), y)
    loss.backward()   # Backward loss propagation
    optimizer.step()  # In-place parameter update
```

Zero Learning Curve

- **1:1 Mental Model**: If you know PyTorch, you already know Sorix.
- **Identical API**: Same classes, training loop, and optimizers.

54 MB Micro-Engine Advantage

- 126× **Lighter**: Full workflow without multi-GB binaries.
- **Pure Transparency**: Tensors and gradients are accessible NumPy/CuPy arrays with zero black boxes.

Thank You for Your Attention!

Спасибо за внимание!

Mitchell Mirano Caro

Personal Portfolio & CV

mitchellmirano.com

Sorix Deep Learning Engine

Documentation & Benchmarks

sorix.mitchellmirano.com

Questions and Comments are Welcome!